```
import os, random, warnings
from pathlib import Path
from collections import Counter, defaultdict

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.auto import tqdm

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler
import torchaudio
import torchaudio.transforms as T

import timm
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, f1_score)
from torch.optim.lr_scheduler import OneCycleLR, CosineAnnealingWarmRestarts
import torch.cuda.amp as amp

warnings.filterwarnings("ignore")
torch.manual_seed(42); np.random.seed(42); random.seed(42)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {DEVICE}")
```

```
Device: cuda
```

```
!pip install -q kaggle

import os
from pathlib import Path

# Kaggle credentials
os.environ['KAGGLE_USERNAME'] = ''
os.environ['KAGGLE_KEY'] = ''

# Download
!kaggle datasets download -d andradaolteanu/gtzan-dataset-music-genre-classification

# Unzip
!unzip -q gtzan-dataset-music-genre-classification.zip -d /content/gtzan

# Paths
DATA_ROOT = Path("/content/gtzan/Data/genres_original")

# Create cache dir
SPEC_CACHE = Path("./spec_cache_10s")
SPEC_CACHE.mkdir(exist_ok=True)

# Load genres
GENRES = sorted([d.name for d in DATA_ROOT.iterdir() if d.is_dir()])
GENRE2IDX = {g: i for i, g in enumerate(GENRES)}
IDX2GENRE = {i: g for g, i in GENRE2IDX.items()}
NUM_CLASSES = len(GENRES)

print("Genres:", GENRES)
```

```
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.0), please consider upgrading to the latest ve
Dataset URL: https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification
License(s): other
gtzan-dataset-music-genre-classification.zip: Skipping, found more recently modified local copy (use --force to force downlo
Genres: ['blues', 'classical', 'country', 'disco', 'hiphop', 'jazz', 'metal', 'pop']
```

```
SAMPLE_RATE     = 22050
N_MELS          = 128
N_FFT           = 2048
HOP_LENGTH      = 512
SEGMENT_DUR     = 10          # seconds — 10s is the sweet spot
SAMPLES_PER_SEG = SAMPLE_RATE * SEGMENT_DUR
IMG_SIZE        = 224

mel_xform = T.MelSpectrogram(sample_rate=SAMPLE_RATE, n_fft=N_FFT,
                             hop_length=HOP_LENGTH, n_mels=N_MELS,
                             f_min=20, f_max=8000, power=2.0)
to_db = T.AmplitudeToDB(top_db=80)
```

```
def load_and_segment(filepath):
    """
    Loads a WAV file → mono 22050Hz → amplitude-normalised →
    slices into non-overlapping 10s mel-spectrogram tensors.
    Returns list of raw-dB tensors (1, N_MELS, T).
    """
    try:
        waveform, sr = torchaudio.load(str(filepath))
    except Exception as e:
        print(f"  Skipping {filepath.name}: {e}")
        return []

    peak = waveform.abs().max()
    if peak < 1e-6:
        return []
    waveform = waveform / (peak + 1e-8)

    if sr != SAMPLE_RATE:
        waveform = T.Resample(sr, SAMPLE_RATE)(waveform)
    if waveform.shape[0] > 1:
        waveform = waveform.mean(dim=0, keepdim=True)   # L+R average → mono

    total = waveform.shape[1]
    segments = []
    for start in range(0, total - SAMPLES_PER_SEG + 1, SAMPLES_PER_SEG):
        seg    = waveform[:, start: start + SAMPLES_PER_SEG]
        mel    = mel_xform(seg)
        mel_db = to_db(mel)          # raw dB, shape (1, 128, ~431)
        segments.append(mel_db)
    return segments

# Pre-compute & cache
print("Pre-computing mel spectrograms …")
all_items = []

for genre in tqdm(GENRES):
    genre_dir = DATA_ROOT / genre
    cache_dir = SPEC_CACHE / genre
    cache_dir.mkdir(exist_ok=True)
    label = GENRE2IDX[genre]
    for wav_file in sorted(genre_dir.glob("*.wav")):
        segs = load_and_segment(wav_file)
        for i, seg in enumerate(segs):
            cp = cache_dir / f"{wav_file.stem}_seg{i}.pt"
            if not cp.exists():
                torch.save(seg, cp)
            all_items.append((str(cp), label))

print(f"Total segments: {len(all_items)}")
label_counts = Counter(item[1] for item in all_items)
print({IDX2GENRE[k]: v for k, v in sorted(label_counts.items())})
```

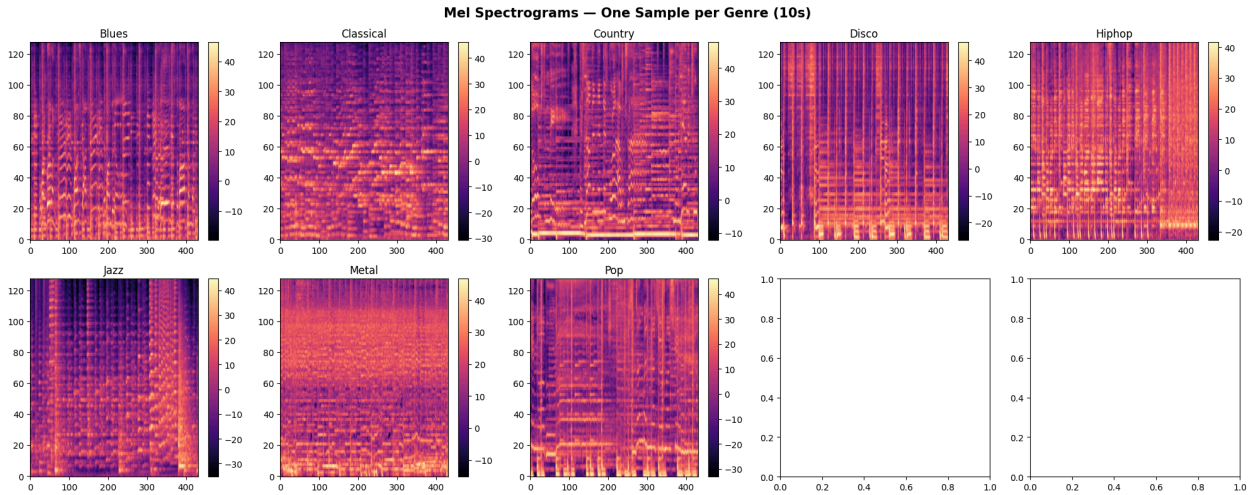
```
Pre-computing mel spectrograms …
100%                                          8/8  [00:41<00:00,   5.03s/it]
  Skipping jazz.00054.wav: Failed to create AudioDecoder for /content/gtzan/Data/genres_original/jazz/jazz.00054.wav: Could
Total segments: 2304
{'blues': 300, 'classical': 298, 'country': 297, 'disco': 299, 'hiphop': 298, 'jazz': 297, 'metal': 300, 'pop': 215}
```

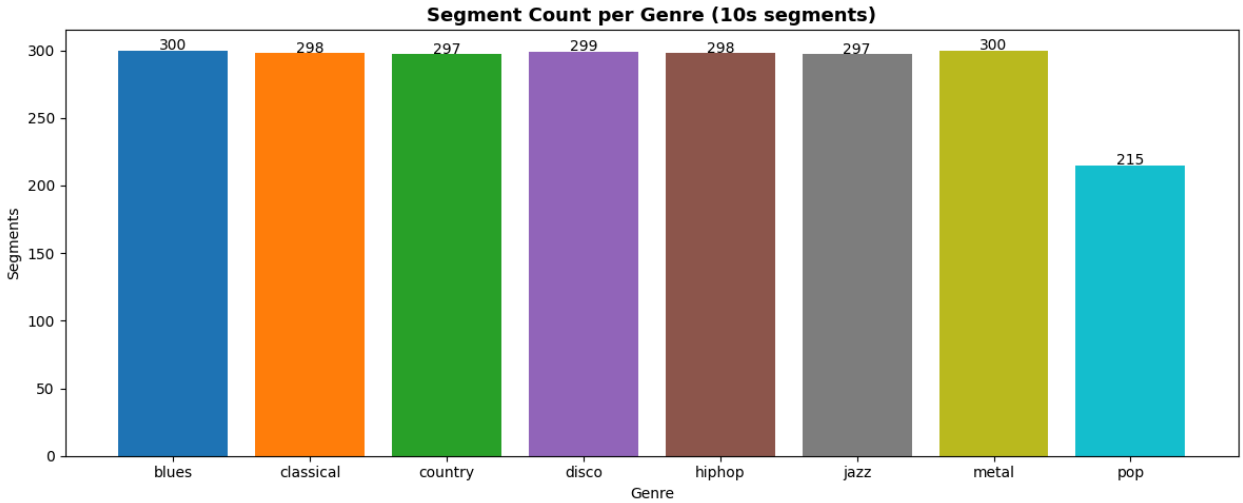
```
print("\nComputing global mel stats …")
running_sum = running_sq = running_cnt = 0.0

for path, _ in tqdm(all_items):
    spec = torch.load(path)
    running_sum += spec.sum().item()
    running_sq  += (spec ** 2).sum().item()
    running_cnt += spec.numel()

GLOBAL_MEAN = running_sum / running_cnt
GLOBAL_STD  = (running_sq / running_cnt - GLOBAL_MEAN ** 2) ** 0.5
print(f"Global mel  mean={GLOBAL_MEAN:.4f}  std={GLOBAL_STD:.4f}")
```

```
Computing global mel stats …
100%                                          2304/2304  [00:01<00:00,  1393.75it/s]
Global mel  mean=8.7164  std=13.1415
```

```
fig, axes = plt.subplots(2, 5, figsize=(20, 8))
fig.suptitle("Mel Spectrograms — One Sample per Genre (10s)", fontsize=15, fontweight='bold')
```

```
for i, genre in enumerate(GENRES):
    sample_path = sorted((SPEC_CACHE / genre).glob("*.pt"))[0]
    spec = torch.load(sample_path).squeeze().numpy()
    im = axes.flatten()[i].imshow(spec, aspect='auto', origin='lower', cmap='magma')
    axes.flatten()[i].set_title(genre.capitalize(), fontsize=12)
    plt.colorbar(im, ax=axes.flatten()[i])
plt.tight_layout()
plt.savefig("eda_melspecs.png", dpi=150, bbox_inches='tight')
plt.show()

fig, ax = plt.subplots(figsize=(12, 5))
bars = ax.bar([IDX2GENRE[k] for k in sorted(label_counts)],
              [label_counts[k] for k in sorted(label_counts)],
              color=plt.cm.tab10(np.linspace(0,1,NUM_CLASSES)))
ax.set_title("Segment Count per Genre (10s segments)", fontsize=13, fontweight='bold')
ax.set_xlabel("Genre"); ax.set_ylabel("Segments")
for b, c in zip(bars, [label_counts[k] for k in sorted(label_counts)]):
    ax.text(b.get_x()+b.get_width()/2, b.get_height()+0.3, str(c), ha='center', fontsize=10)
plt.tight_layout()
plt.savefig("eda_distribution.png", dpi=150, bbox_inches='tight')
plt.show()
```






```
file_to_items = defaultdict(list)
for path, label in all_items:
    stem = Path(path).stem.rsplit("_seg", 1)[0]
    file_to_items[(stem, label)].append((path, label))

file_keys        = list(file_to_items.keys())
labels_for_split = [k[1] for k in file_keys]

train_keys, temp_keys = train_test_split(
    file_keys, test_size=0.2, stratify=labels_for_split, random_state=42)
val_keys, test_keys = train_test_split(
    temp_keys, test_size=0.5, stratify=[k[1] for k in temp_keys], random_state=42)
```

```
def flatten(keys):
    out = []
    for k in keys: out.extend(file_to_items[k])
    return out

train_items = flatten(train_keys)
val_items   = flatten(val_keys)
test_items  = flatten(test_keys)
print(f"Train:{len(train_items)}  Val:{len(val_items)}  Test:{len(test_items)}")
```

```
Train:1839  Val:231  Test:234
```

```
class GTZANDataset(Dataset):
    def __init__(self, items, augment=False):
        self.items   = items
        self.augment = augment

    def __len__(self): return len(self.items)

    def _load(self, path):
        spec = torch.load(path)                      # (1, 128, T)
        spec = F.interpolate(spec.unsqueeze(0),
                             size=(IMG_SIZE, IMG_SIZE),
                             mode='bilinear', align_corners=False).squeeze(0)
        # ── GLOBAL normalisation — identical to Streamlit ──
        spec = (spec - GLOBAL_MEAN) / (GLOBAL_STD + 1e-8)
        spec = spec.repeat(3, 1, 1)                  # (3, 224, 224)
        return spec

    def _spec_augment(self, spec):
        _, F_bins, T_bins = spec.shape
        f  = random.randint(0, 27)
        f0 = random.randint(0, max(F_bins - f, 1))
        spec[:, f0:f0+f, :] = 0
        t  = random.randint(0, 40)
        t0 = random.randint(0, max(T_bins - t, 1))
        spec[:, :, t0:t0+t] = 0
        return spec

    def __getitem__(self, idx):
        path, label = self.items[idx]
        spec = self._load(path)
        if self.augment:
            spec = self._spec_augment(spec)
            if random.random() < 0.5:
                spec = spec.flip(-1)              # time-flip
            if random.random() < 0.3:
                spec = spec + torch.randn_like(spec) * 0.05
        return spec, label


def mixup_batch(specs, labels, alpha=0.3):
    lam = np.random.beta(alpha, alpha)
    lam = max(lam, 1 - lam)
    idx = torch.randperm(specs.size(0))
    return lam * specs + (1-lam)*specs[idx], labels, labels[idx], lam


BATCH_SIZE = 32
train_ds = GTZANDataset(train_items, augment=True)
val_ds   = GTZANDataset(val_items,   augment=False)
test_ds  = GTZANDataset(test_items,  augment=False)

train_labels = [item[1] for item in train_items]
class_counts = Counter(train_labels)
weights      = [1.0 / class_counts[l] for l in train_labels]
sampler      = WeightedRandomSampler(weights, len(weights), replacement=True)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, sampler=sampler,
                          num_workers=2, pin_memory=True, drop_last=True)
val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE, shuffle=False,
                          num_workers=2, pin_memory=True)
test_loader  = DataLoader(test_ds,  batch_size=BATCH_SIZE, shuffle=False,
                          num_workers=2, pin_memory=True)
print(f"Batches — Train:{len(train_loader)} Val:{len(val_loader)} Test:{len(test_loader)}")
```

```
Batches — Train:57 Val:8 Test:8
```

```
class GTZANClassifier(nn.Module):
    def __init__(self, num_classes=10, dropout=0.4):
```

```
        super().__init__()
        self.backbone = timm.create_model('efficientnet_b0', pretrained=True,
                                          num_classes=0, global_pool='avg')
        feat_dim = self.backbone.num_features
        self.head = nn.Sequential(
            nn.BatchNorm1d(feat_dim), nn.Dropout(dropout),
            nn.Linear(feat_dim, 512), nn.GELU(),
            nn.BatchNorm1d(512), nn.Dropout(dropout * 0.75),
            nn.Linear(512, 256), nn.GELU(),
            nn.BatchNorm1d(256), nn.Dropout(dropout * 0.5),
            nn.Linear(256, num_classes),
        )
        for m in self.head.modules():
            if isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight); nn.init.zeros_(m.bias)

    def forward(self, x): return self.head(self.backbone(x))
    def freeze_backbone(self):
        for p in self.backbone.parameters(): p.requires_grad = False
    def unfreeze_backbone(self):
        for p in self.backbone.parameters(): p.requires_grad = True


model = GTZANClassifier(num_classes=NUM_CLASSES).to(DEVICE)
print(f"Params: {sum(p.numel() for p in model.parameters()):,}")
```

```
model.safetensors: 100%                                    21.4M/21.4M [00:02<00:00, 24.3MB/s]
Params: 4,800,900
```

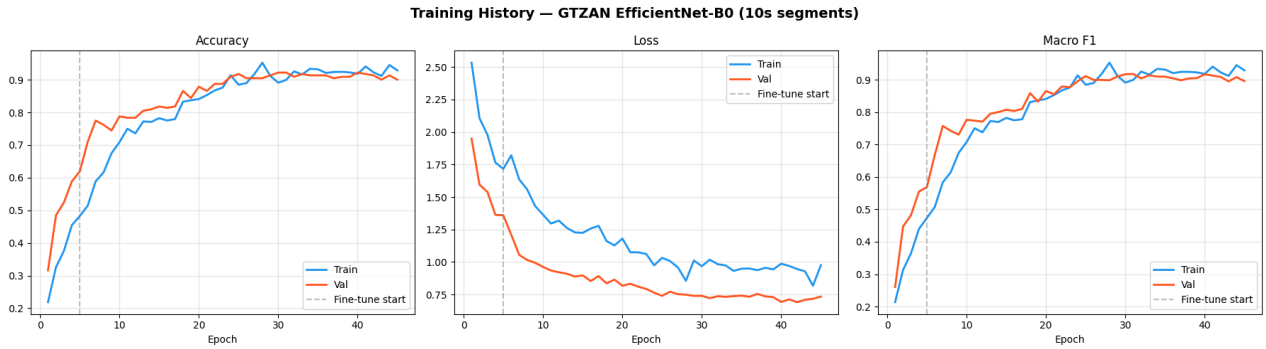
```
scaler    = amp.GradScaler()
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

def mixup_criterion(pred, ya, yb, lam):
    return lam * criterion(pred, ya) + (1-lam) * criterion(pred, yb)

def save_checkpoint(epoch_tag):
    torch.save({
        'model_state': model.state_dict(),
        'genres':      GENRES,
        'genre2idx':   GENRE2IDX,
        'idx2genre':   IDX2GENRE,
        'global_mean': GLOBAL_MEAN,   # ← baked in for Streamlit
        'global_std':  GLOBAL_STD,    # ← baked in for Streamlit
        'val_acc':     best_val_acc,
        'config': {'sample_rate': SAMPLE_RATE, 'n_mels': N_MELS,
                   'n_fft': N_FFT, 'hop_length': HOP_LENGTH,
                   'segment_dur': SEGMENT_DUR, 'img_size': IMG_SIZE}
    }, best_model_path)
    print(f"   ✓ Checkpoint saved ({epoch_tag}  val_acc={best_val_acc:.4f})")

def run_epoch(model, loader, optimizer=None, is_train=True, use_mixup=False):
    model.train() if is_train else model.eval()
    total_loss = correct = total = 0
    all_preds, all_labels = [], []

    with torch.set_grad_enabled(is_train):
        for specs, labels in loader:
            specs, labels = specs.to(DEVICE), labels.to(DEVICE)
            if is_train and use_mixup:
                specs, la, lb, lam = mixup_batch(specs, labels)
            else:
                la = lb = labels; lam = 1.0

            if is_train:
                optimizer.zero_grad()
                with amp.autocast():
                    logits = model(specs)
                    loss   = mixup_criterion(logits, la, lb, lam)
                scaler.scale(loss).backward()
                scaler.unscale_(optimizer)
                nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                scaler.step(optimizer); scaler.update()
            else:
                with amp.autocast():
                    logits = model(specs)
                    loss   = criterion(logits, labels)

            total_loss += loss.item() * labels.size(0)
            preds = logits.argmax(1)
            correct += (preds == la).sum().item()
            total   += labels.size(0)
```

```
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(la.cpu().numpy())

    return total_loss/total, correct/total, f1_score(all_labels, all_preds, average='macro')


WARMUP_EPOCHS   = 5
FINETUNE_EPOCHS = 50
patience        = 15
best_val_acc    = 0.0
best_model_path = "best_gtzan_model.pth"
history = {k: [] for k in
           ['train_loss','val_loss','train_acc','val_acc','train_f1','val_f1']}

# Phase 1 — warm-up
print("="*60 + "\nPHASE 1: Warm-up (head only)\n" + "="*60)
model.freeze_backbone()
optimizer = torch.optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()), lr=3e-3, weight_decay=1e-4)
scheduler = OneCycleLR(optimizer, max_lr=3e-3, steps_per_epoch=len(train_loader),
                       epochs=WARMUP_EPOCHS, pct_start=0.3)

for epoch in range(1, WARMUP_EPOCHS+1):
    tr_loss, tr_acc, tr_f1 = run_epoch(model, train_loader, optimizer,
                                       is_train=True, use_mixup=False)
    scheduler.step()
    vl_loss, vl_acc, vl_f1 = run_epoch(model, val_loader, is_train=False)
    for k, v in zip(['train_loss','val_loss','train_acc','val_acc','train_f1','val_f1'],
                    [tr_loss, vl_loss, tr_acc, vl_acc, tr_f1, vl_f1]):
        history[k].append(v)
    print(f"[WU {epoch:02d}/{WARMUP_EPOCHS}] "
          f"Loss {tr_loss:.4f}/{vl_loss:.4f}  Acc {tr_acc:.4f}/{vl_acc:.4f}")
    if vl_acc > best_val_acc:
        best_val_acc = vl_acc
        save_checkpoint(f"WU{epoch}")

# Phase 2 — full fine-tune
print("\n" + "="*60 + "\nPHASE 2: Full fine-tuning\n" + "="*60)
model.unfreeze_backbone()
optimizer = torch.optim.AdamW([
    {'params': model.backbone.parameters(), 'lr': 1e-4},
    {'params': model.head.parameters(),     'lr': 5e-4},
], weight_decay=1e-4)
scheduler = CosineAnnealingWarmRestarts(optimizer, T_0=10, T_mult=2, eta_min=1e-6)
no_improve = 0

for epoch in range(1, FINETUNE_EPOCHS+1):
    tr_loss, tr_acc, tr_f1 = run_epoch(model, train_loader, optimizer,
                                       is_train=True, use_mixup=True)
    scheduler.step(epoch)
    vl_loss, vl_acc, vl_f1 = run_epoch(model, val_loader, is_train=False)
    for k, v in zip(['train_loss','val_loss','train_acc','val_acc','train_f1','val_f1'],
                    [tr_loss, vl_loss, tr_acc, vl_acc, tr_f1, vl_f1]):
        history[k].append(v)
    improved = vl_acc > best_val_acc
    if improved:
        best_val_acc = vl_acc; no_improve = 0
        save_checkpoint(f"FT{epoch}")
    else:
        no_improve += 1
    tag = "✓ NEW BEST" if improved else f"(no improve {no_improve}/{patience})"
    print(f"[FT {epoch:02d}/{FINETUNE_EPOCHS}] "
          f"Loss {tr_loss:.4f}/{vl_loss:.4f}  Acc {tr_acc:.4f}/{vl_acc:.4f}  {tag}")
    if no_improve >= patience:
        print(f"Early stopping at epoch {WARMUP_EPOCHS+epoch}"); break

print(f"\nBest Val Acc: {best_val_acc:.4f}")
```

```
============================================================
PHASE 2: Full fine-tuning
```

```
[FT 09/50] Loss 1.2280/0.8883  Acc 0.7708/0.8095  ✓ NEW BEST
   ✓ Checkpoint saved (FT10  val_acc=0.8182)
[FT 10/50] Loss 1.2243/0.8970  Acc 0.7823/0.8182  ✓ NEW BEST
[FT 11/50] Loss 1.2573/0.8536  Acc 0.7752/0.8139  (no improve 1/15)
[FT 12/50] Loss 1.2785/0.8917  Acc 0.7796/0.8182  (no improve 2/15)
   ✓ Checkpoint saved (FT13  val_acc=0.8658)
[FT 13/50] Loss 1.1614/0.8363  Acc 0.8328/0.8658  ✓ NEW BEST
[FT 14/50] Loss 1.1277/0.8645  Acc 0.8372/0.8442  (no improve 1/15)
   ✓ Checkpoint saved (FT15  val_acc=0.8788)
[FT 15/50] Loss 1.1800/0.8176  Acc 0.8410/0.8788  ✓ NEW BEST
[FT 16/50] Loss 1.0752/0.8326  Acc 0.8531/0.8658  (no improve 1/15)
   ✓ Checkpoint saved (FT17  val_acc=0.8874)
[FT 17/50] Loss 1.0740/0.8106  Acc 0.8673/0.8874  ✓ NEW BEST
[FT 18/50] Loss 1.0619/0.7930  Acc 0.8761/0.8874  (no improve 1/15)
   ✓ Checkpoint saved (FT19  val_acc=0.9091)
[FT 19/50] Loss 0.9744/0.7645  Acc 0.9139/0.9091  ✓ NEW BEST
   ✓ Checkpoint saved (FT20  val_acc=0.9177)
[FT 20/50] Loss 1.0322/0.7402  Acc 0.8849/0.9177  ✓ NEW BEST
[FT 21/50] Loss 1.0076/0.7720  Acc 0.8898/0.9048  (no improve 1/15)
[FT 22/50] Loss 0.9577/0.7525  Acc 0.9178/0.9048  (no improve 2/15)
[FT 23/50] Loss 0.8556/0.7491  Acc 0.9523/0.9048  (no improve 3/15)
[FT 24/50] Loss 1.0122/0.7395  Acc 0.9117/0.9134  (no improve 4/15)
   ✓ Checkpoint saved (FT25  val_acc=0.9221)
[FT 25/50] Loss 0.9679/0.7401  Acc 0.8909/0.9221  ✓ NEW BEST
[FT 26/50] Loss 1.0180/0.7220  Acc 0.8997/0.9221  (no improve 1/15)
[FT 27/50] Loss 0.9834/0.7371  Acc 0.9260/0.9091  (no improve 2/15)
[FT 28/50] Loss 0.9736/0.7325  Acc 0.9161/0.9177  (no improve 3/15)
[FT 29/50] Loss 0.9316/0.7374  Acc 0.9337/0.9134  (no improve 4/15)
[FT 30/50] Loss 0.9485/0.7416  Acc 0.9320/0.9134  (no improve 5/15)
[FT 31/50] Loss 0.9503/0.7329  Acc 0.9211/0.9134  (no improve 6/15)
[FT 32/50] Loss 0.9374/0.7549  Acc 0.9238/0.9048  (no improve 7/15)
[FT 33/50] Loss 0.9557/0.7362  Acc 0.9243/0.9091  (no improve 8/15)
[FT 34/50] Loss 0.9432/0.7297  Acc 0.9227/0.9091  (no improve 9/15)
[FT 35/50] Loss 0.9870/0.6929  Acc 0.9183/0.9221  (no improve 10/15)
[FT 36/50] Loss 0.9700/0.7126  Acc 0.9408/0.9177  (no improve 11/15)
[FT 37/50] Loss 0.9461/0.6915  Acc 0.9227/0.9134  (no improve 12/15)
[FT 38/50] Loss 0.9285/0.7095  Acc 0.9117/0.9004  (no improve 13/15)
[FT 39/50] Loss 0.8178/0.7174  Acc 0.9452/0.9134  (no improve 14/15)
[FT 40/50] Loss 0.9759/0.7340  Acc 0.9287/0.9004  (no improve 15/15)
Early stopping at epoch 45

Best Val Acc: 0.9221
```

```
epochs_range = range(1, len(history['train_acc'])+1)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("Training History — GTZAN EfficientNet-B0 (10s segments)", fontsize=14, fontweight='bold')
for ax, (tr_k, vl_k), title in zip(
        axes,
        [('train_acc','val_acc'),('train_loss','val_loss'),('train_f1','val_f1')],
        ['Accuracy','Loss','Macro F1']):
    ax.plot(epochs_range, history[tr_k], label='Train', color='#2196F3', lw=2)
    ax.plot(epochs_range, history[vl_k], label='Val',   color='#FF5722', lw=2)
    ax.axvline(WARMUP_EPOCHS, ls='--', color='gray', alpha=0.5, label='Fine-tune start')
    ax.set_title(title); ax.set_xlabel("Epoch"); ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("training_curves.png", dpi=150, bbox_inches='tight')
plt.show()
```



```
ckpt = torch.load(best_model_path, map_location=DEVICE)
model.load_state_dict(ckpt['model_state'])
model.eval()
G_MEAN = ckpt['global_mean']
G_STD  = ckpt['global_std']

def infer_segment(path):
    spec = torch.load(path)
    spec = F.interpolate(spec.unsqueeze(0), size=(IMG_SIZE, IMG_SIZE),
                         mode='bilinear', align_corners=False).squeeze(0)
    spec = (spec - G_MEAN) / (G_STD + 1e-8)
    spec = spec.repeat(3, 1, 1)
    with torch.no_grad(), amp.autocast():
        return torch.softmax(model(spec.unsqueeze(0).to(DEVICE)), 1).squeeze().cpu().numpy()

print("Running test evaluation …")
song_probs = defaultdict(list)
song_true  = {}
for path, label in tqdm(test_items):
    stem = Path(path).stem.rsplit("_seg", 1)[0]
    song_probs[stem].append(infer_segment(path))
    song_true[stem] = label

y_true, y_pred = [], []
for stem in song_true:
    avg = np.mean(song_probs[stem], axis=0)
    y_true.append(song_true[stem])
    y_pred.append(int(np.argmax(avg)))

test_acc = accuracy_score(y_true, y_pred)
test_f1  = f1_score(y_true, y_pred, average='macro')
print(f"\n{'='*50}")
print(f"  TEST ACCURACY: {test_acc:.4f} ({test_acc*100:.2f}%)")
print(f"  TEST MACRO F1: {test_f1:.4f}")
print(f"{'='*50}\n")
print(classification_report(y_true, y_pred, target_names=GENRES))

# Confusion Matrix
for norm, fmt, cmap, title_suffix, fname in [
    (False, 'd',    'Blues',  '',                        'confusion_matrix.png'),
    (True,  '.2f',  'YlOrRd', ' (Normalised)',           'confusion_matrix_norm.png')]:
    cm = confusion_matrix(y_true, y_pred)
    if norm: cm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
    fig, ax = plt.subplots(figsize=(12, 10))
    sns.heatmap(cm, annot=True, fmt=fmt, cmap=cmap,
                xticklabels=[g.capitalize() for g in GENRES],
                yticklabels=[g.capitalize() for g in GENRES],
                linewidths=0.5, ax=ax, vmin=(0 if norm else None), vmax=(1 if norm else None))
    ax.set_title(f"Confusion Matrix{title_suffix}\nAcc: {test_acc*100:.2f}%  F1: {test_f1:.4f}",
                 fontsize=14, fontweight='bold')
    ax.set_xlabel("Predicted"); ax.set_ylabel("True")
    plt.tight_layout(); plt.savefig(fname, dpi=150, bbox_inches='tight'); plt.show()

# Per-class bars
report = classification_report(y_true, y_pred, target_names=GENRES, output_dict=True)
prec = [report[g]['precision'] for g in GENRES]
rec  = [report[g]['recall']    for g in GENRES]
f1s  = [report[g]['f1-score']  for g in GENRES]
x = np.arange(NUM_CLASSES); w = 0.25
fig, ax = plt.subplots(figsize=(14, 6))
ax.bar(x-w, prec, w, label='Precision', color='#4CAF50')
ax.bar(x,   rec,  w, label='Recall',    color='#2196F3')
ax.bar(x+w, f1s,  w, label='F1-Score',  color='#FF5722')
ax.set_xticks(x); ax.set_xticklabels([g.capitalize() for g in GENRES], rotation=30, ha='right')
ax.set_ylim(0, 1.1); ax.legend(); ax.grid(axis='y', alpha=0.3)
ax.set_title("Per-Class Metrics", fontsize=14, fontweight='bold')
plt.tight_layout(); plt.savefig("per_class_metrics.png", dpi=150, bbox_inches='tight'); plt.show()
```